

TEAM PRESENTATION · INTERNAL



A semantic governance gate for your spec files — so nobody, human or AI,  
quietly changes the direction of your project.

Built with Spec Kit · Powered by Claude

## AGENDA

# What we'll cover

---

01

### **The problem**

Specs quietly drift.

---

02

### **The gap**

Existing tools don't guard meaning.

---

03

### **The approach**

SpecGuard, in one picture.

---

04

### **How it works**

One flow, one example.

---

05

### **What's shipped**

Where to try it today.

## PART 1

# Why this exists.

Specs are the source of truth now — and that changes what "a small edit" can mean.

## SPEC-DRIVEN DEVELOPMENT

# Write the spec first. The code follows.

Instead of writing code first and documenting later, you write a plain-language spec — **what** you're building and **why** — and treat it as the source of truth. Humans and AI agents build against it.

By 2026, every major AI coding tool ships an SDD flavor. The category is real.

### THE CATCH NOBODY SOLVED

**If the spec is the source of truth, then whoever edits the spec quietly controls the project's direction.**

And almost nothing guards that.

## SDD ON A REAL TEAM

# The spec is shared — and that's exactly where it gets shaky.

### HOW A TEAM WORKS WITH SDD

-  A few engineers and a PM commit to the same repo.
-  Each engineer pairs with an AI agent — Claude Code, Cursor, Kilo.
-  Everyone reads the same spec — humans *and* agents.
-  Every PR is written *against* that spec.

### WHERE IT GOES WRONG

**Nobody watches every diff. Every agent is stateless.**

One person nudges the spec today. Tomorrow's agent reads the new version and treats it as truth — and starts building against it, no questions asked.



## A SIMPLE SCENARIO

# How a spec drifts.

Meet **Todoly** — a to-do list app for *individuals*. The spec is explicit: *shared team workspaces are out of scope*.

● PR #128 · opened

## "Improve README with usage examples"

```
+ ## Team Collaboration
+
+ Share a workspace with
+ teammates. Assign tasks,
+ comment, and get notified.
```

Author could be a teammate. Or an AI agent asked to *"improve the README."*

## THE PROBLEM

**The title sounds harmless. The diff is five lines.**

But Todoly just quietly turned from a personal to-do app into a team-collab tool — and nobody made that call on purpose.

WHY IT'S EASY TO MISS

# The most dangerous changes aren't in the code.

## **Titles hide direction changes**

"Refactored README for clarity" can carry a complete pivot inside it.

## **Reviewers skim docs**

A tiny Markdown diff rarely gets the scrutiny a code change does.

## **AI agents are stateless**

A confused or poisoned agent can even rewrite the documents meant to govern it.

## **And it compounds**

Each unreviewed shift becomes the new source of truth the next contributor builds on.

SPEC KIT — THE FRAMEWORK MOST TEAMS REACH FOR

# A great workflow — without a lock underneath.

## WHAT SPEC KIT GIVES YOU

A shape for the whole spec workflow.

```
constitution.md → the anchor  
specify → write the spec  
plan · tasks · implement
```

90,000+ stars. Loved by real teams. Excellent at structure.

## WHAT IT DOESN'T GIVE YOU

A way to *enforce* the anchor.

- ✗ No identity check on who edits constitution.md
- ✗ No role-based approval
- ✗ No server-side gate at merge time

"Immutable" is a convention. Any commit can rewrite it — and so can any agent.

*If the source of truth can be edited without checks, it isn't really the source of truth — it's just labeled that way.*



WHAT THE MARKET HAS

# Every tool touches the edges. None guard the meaning.

Spec Kit  
**Structures specs**

No identity, no roles —  
the constitution is  
"immutable" by  
convention only.

OpenSpec  
**Proposal workflow**

Approval = someone  
says *ok* in chat. Checks  
shape, not meaning.

CODEOWNERS  
**Approval by path**

Binary and path-only  
— can't tell a typo from  
a direction change.

**THE GAP**

CODEOWNERS understands file paths. SpecGuard understands **what the change means — and who's allowed to mean it.**

## PART 2

# The approach — SpecGuard.

A governance overlay for the specs you already have. Not a new format.  
Not a replacement.

SPECGUARD, IN ONE LINE

**Lock your project's goal and scope **once**. On every change, ask "**does this move the project somewhere nobody approved?**" — and act on the answer.**

---

**01 · LOCK**

**A locked source of truth**

One small file that says what the project *is*.

---

**02 · CLASSIFY**

**An independent classifier**

An LLM reads each change against the lock — never the same agent that wrote it.

---

**03 · ENFORCE**

**Where it counts**

Advisory everywhere early. Unbypassable at merge time.

# Three verdicts. That's it.



## ADDITIVE

A change within the locked scope — a typo, a clarification, more detail.

---

Passes silently. Zero friction.



## SCOPE\_CHANGE

The change alters the goal or introduces an out-of-scope topic.

---

Warns if unsure — blocks if confident — until an authorized person approves.



## PROTECTED\_VIOLATION

Someone edited a protected file they're not allowed to touch.

---

Hard block. Deterministic — no AI, never a false positive.

WHERE IT CATCHES DRIFT

# Three moments in a change's life.

ADVISORY

WRITE TIME

## Inside the agent

The MCP plugin checks a drafted change *before* the agent writes it. Drift never even lands.



ADVISORY

COMMIT TIME

## On your machine

A pre-commit hook warns you before you push. The commit always succeeds — hooks that block just train people to bypass them.



ENFORCED

MERGE TIME

## In the pull request

The GitHub Action is a required check. Branch protection makes it unypassable. This is the boundary.

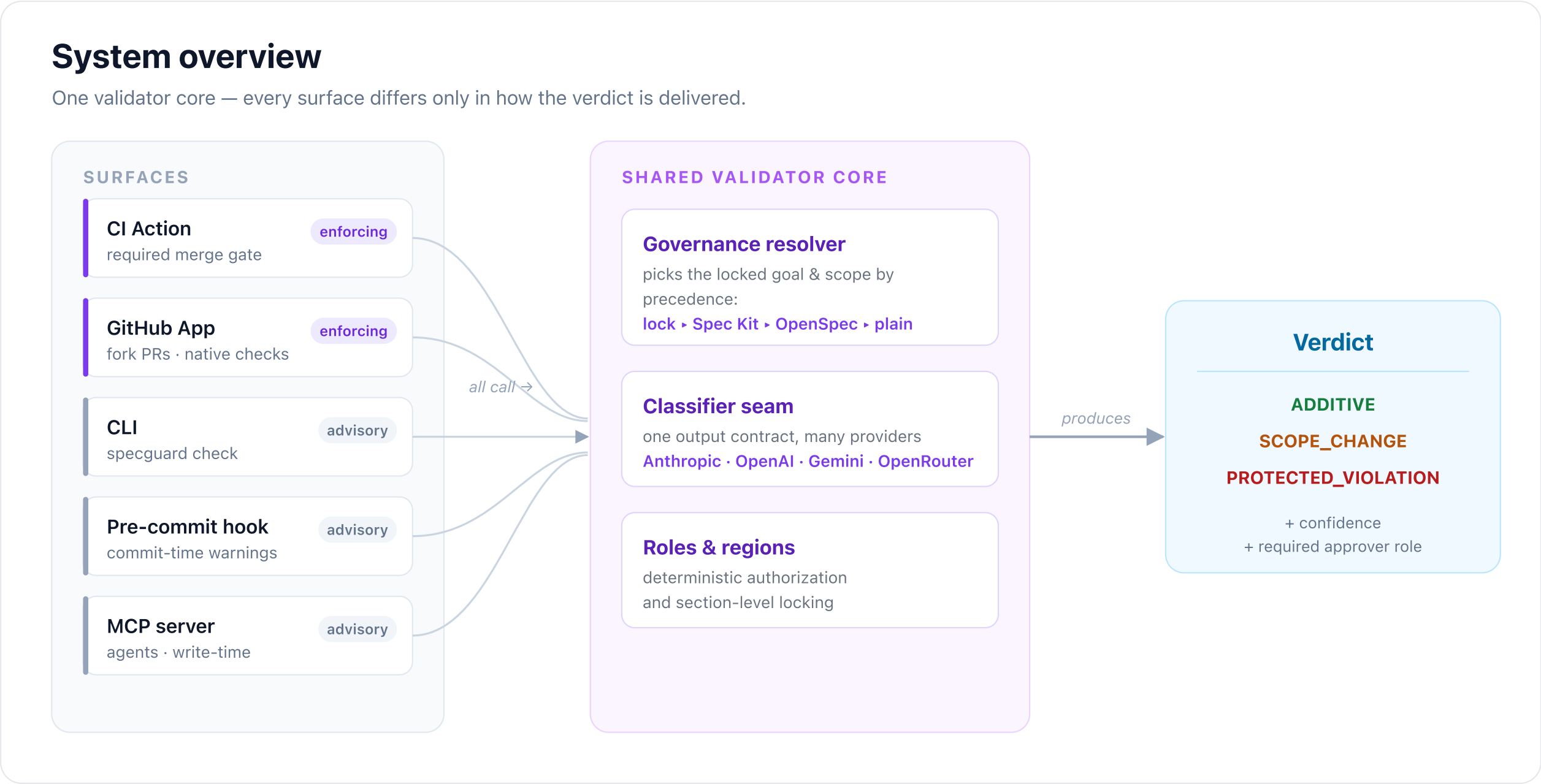
*Catch drift as early and gently as possible — but only **enforce** where it truly can't be worked around.*



ARCHITECTURE

# One core, many surfaces.

A change classified in CI is identical to what the CLI, the hook, or the agent sees. Only the delivery differs.



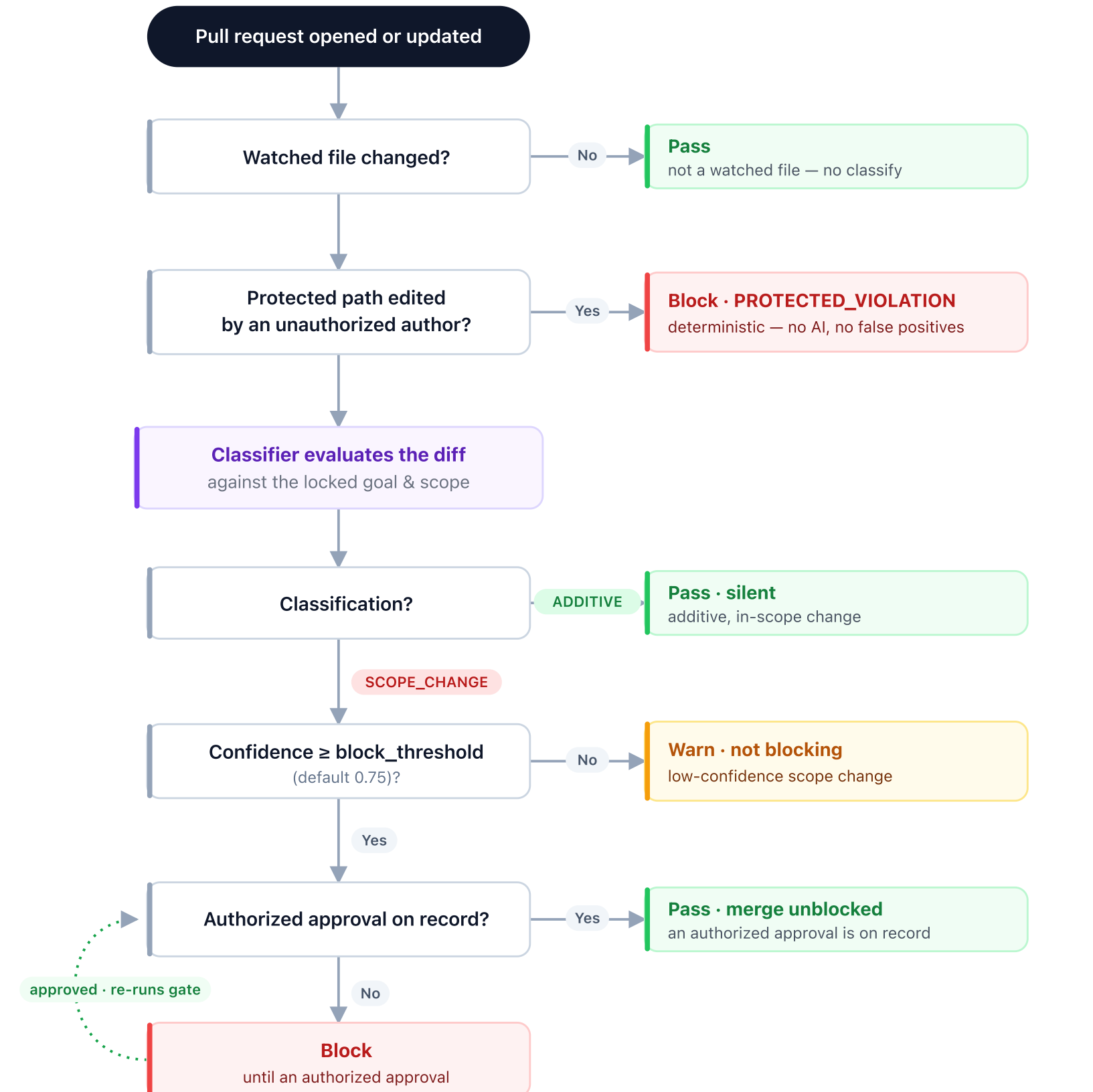
## THE GATE, IN DETAIL

# The decision every change goes through.

- **Not watched?** Passes instantly.
- **Protected file, wrong author?** Hard block, no AI.
- **Watched spec changed?** Classifier reads the diff; confidence decides warn vs block.

### PR gate — decision flow

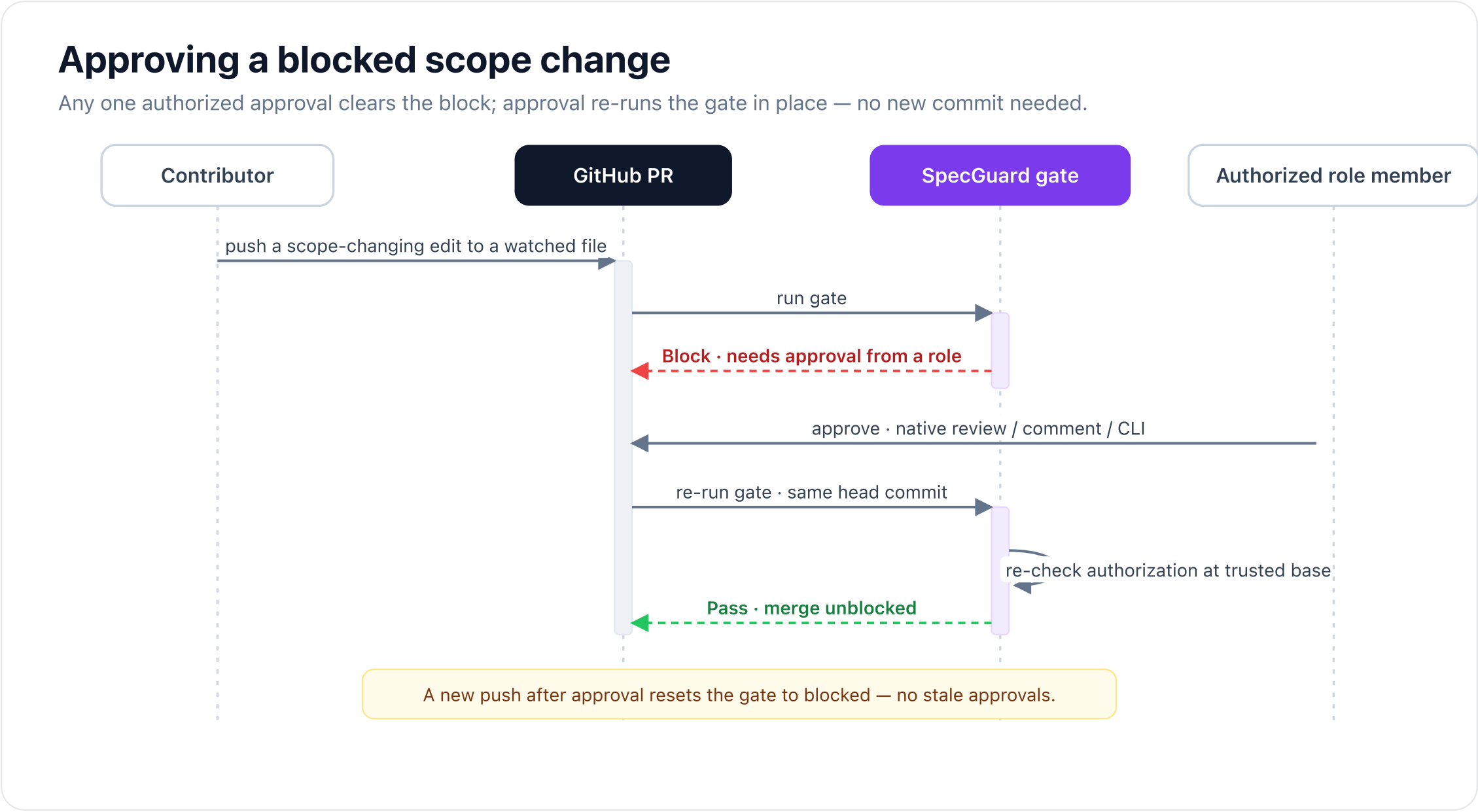
Hard blocks are deterministic (no AI); only a watched-file scope change reaches the classifier.



CLEARING A BLOCK

# One approval. No ceremony.

Any one authorized approval clears the block, and re-runs the gate in place. Three equivalent paths: native PR review, /specguard approve comment, or CLI.





PART 3

# In practice.

The four files that turn it on — and one end-to-end example.

## SETUP

# One command, four files.

```
$ specguard init
```

### lock.json

REQUIRED

#### The source of truth

The project's **goal**, plus **scope\_in** and **scope\_out**. Every change is measured against this one file.

### config.yml

OPTIONAL

#### Behavior

Which files to watch, the block threshold (default 0.75), which model provider to use.

### roles.yml

OPTIONAL

#### Who may approve

Its *presence* is what flips warn into **block**. Without it, scope changes only warn.

### regions.yml

OPTIONAL

#### Lock part of a file

Govern only named sections (a "Goal" paragraph, an "Out of Scope" list). Leave the rest free to edit.

# Todoly, an AI agent, and one PR.

## THE LOCK

Goal

A to-do list app for individuals to manage their own tasks.

Explicitly out of scope

**team workspaces · shared assignments · admin roles ·  
realtime collab**

---

Architect: @Leandro — the only one who can approve a scope change.

## THE PROMPT

"Improve the README with usage examples."

The AI agent decides to add a **Team Collaboration** section.  
Let's follow that change through all three moments.

THE CHANGE, CAUGHT THREE WAYS

# One diff. Three chances to stop it.

## ① WRITE TIME

### The agent self-corrects

```
agent →  
check_proposed_change(  
  file="README.md", ...)  
SpecGuard → SCOPE_CHANGE  
confidence 0.92  
out-of-scope: team workspaces
```

The agent sees the redirect and **backs off**. Drift is stopped before it exists.

## ② COMMIT TIME

### A friendly heads-up

```
$ git commit -m "expand  
README"  
  
specguard ⚠ SCOPE CHANGE  
  topic: team workspaces  
  will need @leandro to  
  approve  
  
[main 4f2a1c9] commit  
succeeds
```

Warned before pushing — but the commit is **never blocked**. No surprise later.

## ③ MERGE TIME

### The real boundary

```
✗ specguard — Changes  
requested  
  SCOPE CHANGE · 92%  
confidence  
  topic: team workspaces  
  requires approval: @leandro  
Merge blocked 🛑  
  (required check: specguard)
```

The merge button is disabled by branch protection. **This layer can't be bypassed.**

## RESOLUTION

# Now there are only two honest outcomes.

### A · IT WAS A MISTAKE

#### The section is dropped.

Todoly stays a personal to-do app. The gate turns green on the next push.

### B · IT WAS INTENTIONAL

#### @leandro approves.

The gate re-runs in place. Merge unblocks.



The point isn't that team collaboration is *bad*. It's that the direction change was **caught, explained, and decided by the right person on purpose** — instead of slipping in through a five-line diff nobody read.



STATUS

# Where it stands today.

|           |                                                                             |                 |
|-----------|-----------------------------------------------------------------------------|-----------------|
| PHASE 0   | CI gate — Action · scope classification · role approval · branch protection | ● Shipped       |
| PHASE 1   | Local tools — CLI · pre-commit hook · MCP plugin                            | ● Shipped       |
| PHASE 1.5 | Provider-agnostic — Anthropic · OpenAI · Gemini · OpenRouter                | ● Shipped       |
| PHASE 2   | Framework adapters · approval commands (/specguard approve)                 | ● Shipped       |
| PHASE 2   | GitHub App — fork PRs · native check runs · bot-vs-human identity           | ● Code-complete |
| PHASE 3   | Advanced — section-level locking · monorepo multi-scope · audit export      | ● Shipped       |

Calibration: Anthropic + Sonnet 4.6 is the verified default — **0 false blocks, ≥90% recall** on the golden corpus. Other providers wired & tested, not yet live-calibrated.

IN ONE BREATH

# Lock the goal. Let additive edits pass silently. Explain and block real direction changes at merge time.

---

## NO FALSE BLOCKS

Additive edits pass silently. Confidence is always shown.

---

## NO NEW UI

Lives in the repo and the PR. No dashboard to log into.

---

## ONE BOUNDARY

Merge time is the only enforceable line. Everything else helps you get there cleanly.

# Thank you.

Try it

`pip install specguard-ci`  
GitHub Marketplace · SpecGuard CI